

DEAKIN UNIVERSITY

FULL STACK DEVELOPMENT: SECURE FRONTEND APPLICATIONS

ONTRACK SUBMISSION

Subscription

Submitted By:

Romil BIJARNIA

s222528574

2026/09/18 02:52

Tutor:

Nisha KUMARI

Task Outcomes		Supports
TLO1	Demonstrate engagement with the following unit learning outcomes (legacy)	ULO Unit Learning Outcome 1

September 17, 2026



Subscription

Romil Bijarnia | Student ID 222528574

DEV@Deakin combines validated registration and login with signed user sessions and a Free/Paid membership. The Paid upgrade is a demonstration: it saves the account plan but takes no payment.

Repository: <https://gitlab.deakin.edu.au/s222528574/task-d1/-/tree/review-d1-20260906/reviewed-app>

Hosted application: <https://sit313-d1-draft--dev-deakin-sit313-romil.netlify.app>

Video: [Panopto walkthrough \(5:17\)](#)

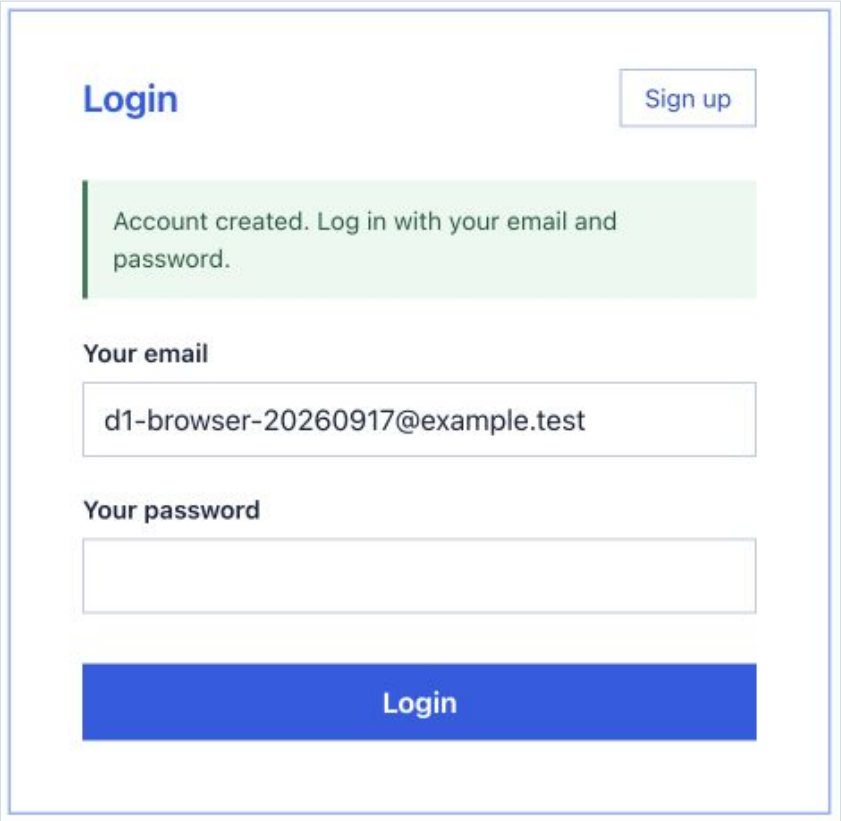
Registration and password checking

Client and server validation check the account fields and matching passwords. The server hashes the password with `bcrypt` before creating a Firestore user on the Free plan. A successful registration returns to Login; it does not automatically sign in. Login retrieves the account and checks the supplied password with `bcrypt.compare`. Invalid credentials receive the same failure message.

Signed sessions and logout

A successful login returns a signed JWT, stored in `localStorage`. Protected requests carry it as a Bearer token. Express checks the signature, issuer, audience and expiry, then loads the account from Firestore. The current plan comes from that account, not a client-supplied plan value.

Logout clears the local token immediately and requests server-side session revocation. If revocation cannot be confirmed, a warning remains visible while this browser stays signed out.



The screenshot shows a web interface for a login page. At the top left, the word "Login" is displayed in a blue font. At the top right, there is a "Sign up" button. Below the header, a green message box states: "Account created. Log in with your email and password." Underneath this, there are two input fields. The first is labeled "Your email" and contains the text "d1-browser-20260917@example.test". The second is labeled "Your password" and is currently empty. At the bottom of the form, there is a large blue button labeled "Login".

Figure 1. The new demonstration account is created and returned to Login.

Plan comparison, eligibility and validation

The Pricing page compares Free and Paid plans. Prices and listed allowances are illustrative. Only a signed-in Free account can open the upgrade form; visitors must log in and an existing Paid account cannot upgrade again.

FOR EVERYDAY LEARNING

Free

\$0 / month

Read, ask questions and share what you are learning.

Your current plan

PAID MEMBERSHIP

Paid

\$9 AUD / month - illustrative

A demonstration of a membership with more room to contribute.

Upgrade Plan

No charge. Demonstration only.

Figure 2. The new account begins on Free, with Upgrade Plan available.

Upgrade Plan

[Log in](#) to upgrade your plan.

Figure 3. For a signed-out visitor, the upgrade control is disabled and Login is offered.

The modal checks the cardholder name, the displayed test card, a valid expiry, a three-digit test code and confirmation that the upgrade is simulated. No real card should be used.

Demonstration card number

Use the demonstration card 4242 4242 4242 4242. Do not enter a real card.

Expiry (MM/YY)

Use MM/YY.

Test security code

Enter a three-digit demonstration security code.

Figure 4. Missing or invalid payment-demonstration fields produce specific errors.

Successful upgrade and restored state

The upgrade request validates the form and rechecks the authenticated account. A Firestore transaction changes Free to Paid atomically, preventing a second concurrent upgrade. The stored changes are the plan, server upgrade time and simulation marker; payment fields do not reach the persistence method.

Your plan is now Paid. This was a demonstration; no payment was taken.

Figure 5. The server confirms the successful plan change and the interface states that no payment was taken.

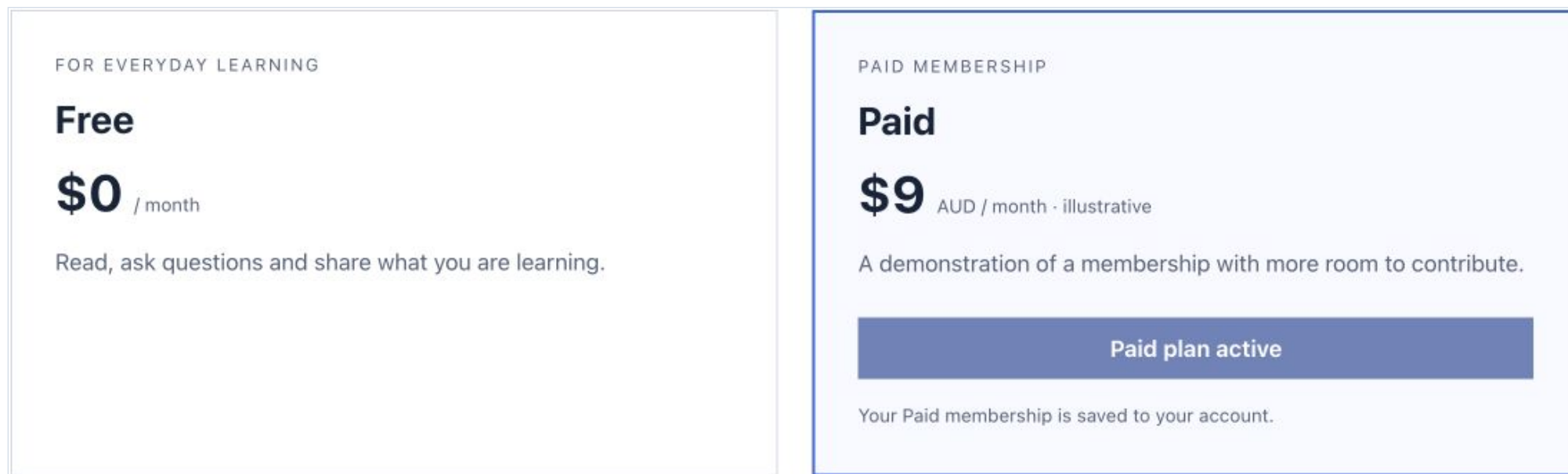


Figure 6. After reloading the hosted application, Paid remains active and the upgrade button is unavailable.

On restoration, the application sends the saved JWT to the session endpoint and displays the latest Firestore account state. A plan change therefore survives page reloads instead of depending on a temporary React state value.

Verification. 113 automated checks passed, including invalid authentication, session races, duplicate upgrades and storage failures. [GitLab pipeline 8460](#) passed for the published source.

Saved user in Firestore

The walkthrough account `d1-video-20260917-02@example.test` appears in `sit313_d1_users` with plan `Paid` and `upgradeMode` simulation. The saved plan and server upgrade time confirm the subscription change demonstrated in the recording.

Collection *

/sit313_d1_users

Query results

Document ID	email	plan	upgradeM	
b248ffb8694358326427bb90a7d4c58a560f8be466846db9d31a9f41678f4b30	"d1-video-20260917-02@example.test"	"Paid"	"simulation"	View all row content

Figure 7. Google Cloud Firestore shows the matching walkthrough account and its Paid membership.

The database stores a password hash, not a plain-text password. Card number and security code are not saved. Backend validation and session checks apply before any protected account or subscription operation.